

Set 4

Here are the instructions for each exercise in **Set 4**.

You can create a workspace for your **Set4** folder in **VS Code** by selecting **Open Folder...** and choosing your **Set4** folder. This allows you to access and manage all your exercise files in one place.

Open the **.c** file you want to run using **VS Code** or any other text editor you prefer (for example, Notepad on Windows). You can edit your C file directly in any of these editors.

When you finish writing your code, you need to **compile it**. To do this, open **w64devkit** and navigate to the **exercise folder** you are working on within your **Set4** directory. For detailed steps on how to navigate to the folder and compile your program, refer to the *"Getting started with LEU483"* guide on Canvas.

You don't need to do all exercises at once, some can be saved to practice for the exam. However, it is recommended that you at least finish exercises 1-6 before starting to work on assignment 2a and 7-9 before working on assignment 2b.

1 Sum array

Open: `ex_sum_arr/main.c`

Samples: `sample_arrays`

In this exercise you will calculate the sum of the elements in an array.

Task:

- Initialize an array with some integer values e.g., the values 2, 4, 5, 6, 3.
- Use `sizeof` to get the size of the array. Note that `sizeof(arr)` will give you the size of the array in bytes. To get the size of the array as number of elements you need to divide by the size of an element in the array.
- Print the size of the array and check that it is correct. In the example of values 2, 4, 5, 6, 3 the size should be 5.
- Print the element values of the array.
- Calculate the sum of the elements in the array and print the result.

Example output:

```
Size of array is: 5
Elements of the array are: 2 4 5 6 3
Sum of elements is: 20
```

2 Basic array functions

Open: `ex_arr_basic/main.c`

Samples: `sample_arrays`, `sample_arr_func`

As with ordinary variables in C it is important that also arrays are properly initialized so that no unexpected values that are stored in the array affect the result when the arrays are used. In this exercise you will implement two functions that can be very useful when handling arrays. One that initializes all elements in an array to a specific value and one that prints the values of an array and allows the programmer to check that the array doesn't store unexpected values.

Task:

- Implement a function called `init_arr` that assigns each element in an integer array with an arbitrary value (the value should be passed as an argument to the function).
- Implement a function called `print_arr` that prints the value of each element in an integer array.
- Use `init_arr` to initialize the array `intarray` with ones.
- Use `print_arr` to print the values of `intarray`.
- Make sure that your functions work for arrays with arbitrary size i.e., the size of the array should be passed as an argument to the functions.

Program Output

```
Elements of the array are: 1 1 1 1 1 1
```

3 Matrix

Open: ex_matrix/main.c

Samples: sample_matrix

This is a program that lets the user input a small matrix and then prints the matrix and the transpose of the matrix. The values of the user-defined elements are stored in the 2D-array called `matrix`.

Task:

- Transpose the values in `matrix` and store the result in the 2D-array `transpose`.
- Print the original matrix.
- Print the transpose of the matrix.

Example output:

```
Program to transpose matrix
Input a 2 x 3 matrix
Input 0 0 > 1
Input 0 1 > 2
Input 0 2 > 3
Input 1 0 > 4
Input 1 1 > 5
Input 1 2 > 6
Matrix is
1 2 3
4 5 6
Transpose is
1 4
2 5
3 6
```

4 Basic 2D array functions

Open: `ex_2d_arr_basic/main.c`

Samples: `sample_matrix`, `sample_matrix_func`

This exercise is similar to exercise 2. You will implement the same functions but for 2D arrays.

Task:

- Implement a function called `init_2Darr` that assigns each element in an integer 2D array with an arbitrary value (the value should be passed as an argument to the function).
- Implement a function called `print_2Darr` that prints the value of each element in an integer 2D array. The values should not be printed in one row, the structure of the 2D array should be shown. See program output below.
- Use `init_2Darr` to initialize the array `int2Darray` with ones.
- Use `print_2Darr` to print the values of `int2Darray`.
- Make sure that your functions work for 2D arrays with arbitrary number of rows and columns i.e., the number of rows and columns should be passed as arguments to the functions.

Program output:

```
Elements of 2D array with 2 rows and 3 columns are:  
1 1 1  
1 1 1
```

5 Enum

Open: `ex_enum/main.c`

Samples: `sample_enum`

In this exercise, you will demonstrate the use of `enum` and be introduced to the use of `typedef`.

Task:

- Define an enumeration named `days`, containing values corresponding to the days of the week.
- Initialize a variable called `today` that holds the current day
- Use your enumeration to check against the variable `today`, to see if today is the weekend.
- Test that the code still works when setting `today` to a different day (e.g. Sunday).
- Rename the type `enum days` to `days_t` using `typedef` and initialize the variable `today` using `days_t` instead. The output of the program should be the same.

Program output:

```
It's not the weekend yet - keep working on your assignment!
```

6 Christmas tree with enum

Open: `ex_xmastree_enum/main.c`

Samples: `sample_enum`

This exercise is similar to the exercise Christmas tree in set 4, but this time you will define an enumeration to contain the various characters that make up the tree and its decorations.

Task:

- Define an enumeration with items: `tree`, `decoration`, `decoration_top`, and `trunk`. Specify the values of these items with the characters that correspond to each item. (You do not need to ask for a user input in this exercise.)
- Name the enumeration `tree_parts_t` using `typedef`.
- Define a function that draws a decorated christmas tree and one that draws the trunk of the tree.
- The functions should take the relevant tree parts from the enumeration `tree_parts_t` as arguments.

Hint: Look at your solution of `set3/ex_xmas_tree/main.c`, and change it accordingly. Take note that you should also include the `decoration_top` in your revised code.

Program output:

```
  +
 0*0
0***0
0*****0
0*****0
0*****0
  |
  |
```

7 Students

Open: `ex_students/main.c`

Samples: `sample_struct`

This is a program to keep track of student grades. Each student is represented by a structure that holds the social security number of the student and the students grade. In this example we have a very small class with only 3 students. The program asks the user for student information (social security number and grade) and then stores this information in an array of structures. The program also asks the user for a student number and views the information for that student.

Task:

- Define a structure called `student` which has two members. One of type `unsigned long long int` called `pnumber` and one of type `char` called `grade`.
- Declare an array to store the student information in. This array should have one element for each student, and it should store student structures.

- In the for-loop, declare a variable called `stud` of type `structure student` to hold the information provided by the user.
- Assign the input values to the members of `stud`.
- Store the structure `stud` in the array `grades`.
- Ask the user for which student they want to see the stored information and print it.

NOTE: this program does not include any checks of the input from the user which is bad practice. What happens for instance if the user enters a student number that is larger than the number of students?

Example output:

```
Student 1
Enter social security number: 200202027835
Enter grade (A-F): A
Student 2
Enter social security number: 200001156348
Enter grade (A-F): F
Student 3
Enter social security number: 199910169843
Enter grade (A-F): C

Enter student number to see social security number and grade: 2
social security number: 200001156348
grade: F
```

8 typedef for structures

Open: `ex_students/main.c`

Samples: `sample_struct`

This is a small exercise demonstrating the use of `typedef` for structures. In this exercise you will make use of `typedef` to rewrite the code in exercise 7 to make it a bit cleaner.

Task:

- Use `typedef` to rename the type `unsigned long long int` to `ulong`.
- Use this new name in the rest of the code.
- Use `typedef` to rename structure to `student_t`.
- Use this new name in the rest of the code.

The output of the program should be the same as in exercise 7.

9 Triangles

Open: `ex_pts_tri/main.c`

Samples: `sample_struct`, `sample_struct_fun`

In this exercise you work with points and triangles in 3D. Both are represented as structures.

Task:

- Implement the function `distance` to calculate the distance between two points.
- Implement the function `area` to calculate the area of a triangle. Use Herons formula in the calculation of the area, see https://en.wikipedia.org/wiki/Heron%27s_formula. You should also use the function `distance` to get the length of each side of the triangle.
- The final output should print ok for all tests.

NOTE: Structures are always passed by address to functions (i.e. a pointer to the structure).

NOTE: When you use functions from `math.h`, remember to link the math library by adding `-lm` at the end of your compile command.

10 Array functions

Open: `ex_arr_func/main.c`

Samples: `sample_arrays`, `sample_arr_func`

Task:

- Implement each function one at the time and make sure they pass the tests.
- The final output should print ok for all except the three last which should output a random reordering of the array (permutation).

NOTE 1: Use the function `round` from `math.h` in the implementation of `distribute`. Take note to add the flag `-lm` at the end of your compile command, to link the math library. Revisit quadratic function exercise in Set 2.

NOTE 2: The function `shuffle` should use the modern version of the Fisher-Yates algorithm to do the reordering. See: https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle.

11 Matrix functions

Open: `ex_matrix_func/main.c`

Samples: `sample_matrix`, `sample_matrix_func`

In this exercise you will implement some functions for matrices that are represented by 2D arrays.

Task:

- Implement each function and make sure they pass the tests.
- The final output should print ok for all 11 tests.

NOTE: The function `is_valid_location` can be useful in the implementation of `sum_neighbours`.

NOTE: When you use functions from `math.h`, remember to link the math library by adding `-lm` at the end of your compile command.

12 Remove duplicates

Open: `ex_remdupl/main.c`

In this exercise you will implement the function `remove_dupl` that removes duplicates in a **sorted** integer array.

Task:

- The destination array shall have the same size as the source array, pad with 0's.
- You can assume that the input array is sorted.
- Make sure that all tests pass. The final output should print ok for both tests.

13 Median

Open: `ex_median/main.c`

In this exercise you will implement two different sorting algorithms to sort an array with integers such that the median value of the array can be calculated.

Task:

- Implement the function `is_sorted` that checks if an integer array is sorted.
- Implement the function `bubble_sort` that uses the bubble sort algorithm to sort an integer array. See: https://en.wikipedia.org/wiki/Bubble_sort.
- Implement the function `insertion_sort` that uses the insertion sort algorithm to sort an integer array. See https://en.wikipedia.org/wiki/Insertion_sort.
- Implement the function `median` that calculates the median of a sorted integer array.
- The final output of the program should be as below if your implementations are correct.

Program output

```
ok
1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 6, 7, 8, 8, 9, 9
ok
ok
ok
1, 2, 3, 3, 4, 4, 5, 5, 6, 6, 6, 7, 7, 8, 9
Size of arr is 16 median is 4.500000
Size of arr2 is 15 median is 5.000000
```